

QVerse — Week 5: Multiple Qubits and Qiskit Bit Ordering

Theory Booklet | Introduction to Quantum Computing & Programming

QVerse

2026

How this booklet fits the course

This theory booklet is designed to be read alongside `05_multiple_qubits_and_bit_ordering.ipynb`. It contains the conceptual and mathematical material that should be explained during Week 5; the notebook then turns those ideas into experiments, circuit construction, and exercises.

Learning objectives

- Enumerate two-qubit computational basis states.
- Understand that n qubits require 2^n amplitudes.
- Prepare product states and product superpositions.
- Interpret Qiskit's qubit/bit ordering correctly.

Recommended workflow

1. Read the relevant theory section.
2. Predict the result of the corresponding notebook experiment.
3. Run the notebook.
4. Explain any difference between the prediction and the output.
5. Complete the core exercises without treating the solution notebook as a substitute for reasoning.

1. Moving from one qubit to many

A one-qubit state requires two amplitudes. A two-qubit state requires four. In general, an n -qubit pure state requires

$$2^n$$

complex amplitudes.

This exponential growth is one reason quantum systems are difficult to simulate classically. It is also why notation and bit ordering must be handled carefully.

2. Two-qubit computational basis

The computational basis for two qubits is

$$|00\rangle, \quad |01\rangle, \quad |10\rangle, \quad |11\rangle.$$

A general two-qubit pure state is

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle,$$

with normalization

$$|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1.$$

3. Tensor products

If qubit A is in

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

and qubit B is in

$$|\phi\rangle = \gamma|0\rangle + \delta|1\rangle,$$

the joint product state is

$$|\psi\rangle \otimes |\phi\rangle.$$

Expanding:

$$|\psi\rangle \otimes |\phi\rangle = \alpha\gamma|00\rangle + \alpha\delta|01\rangle + \beta\gamma|10\rangle + \beta\delta|11\rangle.$$

The symbol $\otimes$ is the tensor product.

For the beginner-level workflow, it is enough to understand that the tensor product combines subsystem descriptions into one larger state space.

4. Product states

A multi-qubit state is a **product state** if it can be written as a tensor product of individual subsystem states. For example,

$$|+\rangle \otimes |1\rangle = \frac{|01\rangle + |11\rangle}{\sqrt{2}}$$

under a particular abstract qubit-label ordering.

A product state can still contain several nonzero amplitudes. “Multiple possible outcomes” does not automatically mean entanglement.

Entanglement, introduced in Week 6, occurs when the joint state cannot be factored into independent single-qubit states.

5. State-space scaling

The number of basis states is

$$N = 2^n.$$

Examples:

Qubits n	Amplitudes 2^n
5	32
10	1,024
20	1,048,576
30	1,073,741,824

A full statevector simulator must store all these complex amplitudes.

This does **not** mean that a quantum computer automatically provides an exponential speedup. It means the mathematical state space is exponentially large. A useful algorithm must still manipulate and extract information efficiently.

6. Qubit labels versus displayed bit strings

This is one of the most important practical details in Qiskit.

Qiskit usually labels qubits as

$$q_0, q_1, \dots, q_{n-1}.$$

When a classical bit string is displayed, the highest-index classical bit is conventionally shown on the **left**. Therefore the printed order often appears reversed relative to the order in which beginners list qubit indices.

For a two-qubit example:

- q_0 corresponds to the rightmost displayed bit;
- q_1 corresponds to the leftmost displayed bit.

The safest rule is not to memorize a sentence in isolation. Prepare a known basis state and inspect the displayed result.

7. A concrete ordering check

Suppose a two-qubit circuit starts in $|00\rangle$ and applies X to q_1 .

Physically, qubit 1 is set to 1 and qubit 0 remains 0.

In Qiskit's standard displayed bitstring convention, the measurement result is shown as

$$10.$$

Applying X only to q_0 gives displayed result

$$01.$$

This convention becomes critical when implementing oracles. A silent bit-order reversal can mark the wrong computational basis state while the code still runs successfully.

8. Statevector basis ordering

Statevector arrays also follow a basis-index convention. For a two-qubit system, the four amplitudes correspond to basis-state labels associated with binary indices.

Instead of assuming the array ordering, use tools such as a probability dictionary when learning:

- it exposes human-readable basis labels;
- it helps connect qubit preparation with printed strings;
- it reduces indexing mistakes.

Later, when performance matters, the array representation becomes more useful.

9. Preparing basis states

Any computational-basis bit string can be prepared from $|00\dots 0\rangle$ by applying X to the qubits that should contain a 1.

Conceptually, for a target string $b_{n-1} \dots b_1 b_0$,

$$|00\dots 0\rangle \rightarrow |b_{n-1} \dots b_1 b_0\rangle$$

by applying

$$X^{b_j}$$

to qubit q_j .

This naturally motivates a helper function that accepts a bit string and builds the corresponding circuit.

10. Preparing product superpositions

Independent Hadamard gates can create product superpositions.

For example,

$$H|0\rangle = |+\rangle.$$

If one qubit is placed in $|+\rangle$ while another remains in a basis state, the joint state may contain multiple basis components while still being separable.

For two independent plus states,

$$|+\rangle|+\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle).$$

All four outcomes have equal probability $1/4$, but the state is not entangled because it factors as $|+\rangle \otimes |+\rangle$.

11. Multi-qubit gates preview

A multi-qubit system allows gates that couple qubits. The most important first example is the controlled- X or CNOT gate, introduced fully in Week 6.

The distinction to remember is:

- single-qubit gates transform one subsystem locally;
- entangling two-qubit gates can create correlations that cannot be represented as independent states.

12. Exponential state space and simulation

A rough memory estimate illustrates the classical simulation problem. If each complex amplitude uses 16 bytes, a dense n -qubit statevector requires approximately

$$16 \cdot 2^n \text{ bytes.}$$

For $n = 30$,

$$16 \cdot 2^{30} \approx 16 \text{ GiB.}$$

At $n = 40$, this becomes about 16 TiB.

Specialized simulation methods can exploit structure in some circuits, but the generic dense statevector scaling remains exponential.

13. Common misconceptions

“Several nonzero amplitudes imply entanglement”

No. Product superpositions can contain many terms.

“Qiskit changed the qubit values because the printed string looks reversed”

Usually the issue is display convention, not a physical transformation.

“The exponential statevector means an exponential speedup is guaranteed”

No. Quantum advantage depends on the algorithm and the problem structure.

“Tensor product means ordinary vector addition”

No. It creates a larger composite state space.

14. What to know before the notebook

You should be able to:

- list the four two-qubit basis states;
- state that n qubits use 2^n amplitudes;
- distinguish product states from the idea of entanglement;
- expand a simple tensor product;
- explain Qiskit's printed bit-order convention with a concrete example.

Suggested references

- IBM Quantum Learning, *Basics of Quantum Information* and current Qiskit learning material: <https://quantum.cloud.ibm.com/learning/en/courses>
- IBM Quantum Documentation and Qiskit guides: <https://quantum.cloud.ibm.com/docs>
- Quantum Rings, *Quantum 101*: <https://www.quantumrings.com/course/quantum101>
- M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press.

The software-facing details in the practical notebook should be treated as the canonical implementation for this course. The purpose of this booklet is to explain the physics, mathematics, and algorithmic reasoning behind that implementation.